



**Bahria University Islamabad**

Department of Computer Science

Digital Image Processing — 7th Semester

---

# Deepfake Detection System

*Using Frame-Based Image Classification*

---

**Final Project Report**

**Submitted by:**

Laiba Tauseef                      01-134231-033

Syeda Maleeha Bano Naqvi   01-134231-092

**Submitted to: Sir Muhammad Siddique**

Department of Computer Science

Bahria University Islamabad

May 12, 2026

# Contents

|                                               |           |
|-----------------------------------------------|-----------|
| <b>Abstract</b>                               | <b>3</b>  |
| <b>1 Introduction</b>                         | <b>4</b>  |
| 1.1 Background . . . . .                      | 4         |
| 1.2 Problem Statement . . . . .               | 4         |
| 1.3 Project Objectives . . . . .              | 4         |
| 1.4 Scope and Limitations . . . . .           | 5         |
| 1.4.1 What the System Does . . . . .          | 5         |
| 1.4.2 What the System Does Not Do . . . . .   | 5         |
| <b>2 Literature Review</b>                    | <b>6</b>  |
| 2.1 Deepfake Generation Techniques . . . . .  | 6         |
| 2.2 Deepfake Detection Approaches . . . . .   | 6         |
| 2.2.1 CNN-Based Detection . . . . .           | 6         |
| 2.2.2 Transfer Learning Approaches . . . . .  | 6         |
| 2.2.3 Temporal Analysis . . . . .             | 6         |
| 2.2.4 Face-Region Focused Detection . . . . . | 7         |
| 2.3 Selected Approach Justification . . . . . | 7         |
| <b>3 Methodology</b>                          | <b>8</b>  |
| 3.1 Dataset . . . . .                         | 8         |
| 3.2 Data Preprocessing . . . . .              | 8         |
| 3.3 Data Augmentation . . . . .               | 8         |
| 3.4 Model Architecture . . . . .              | 9         |
| 3.4.1 Base Model: EfficientNetB0 . . . . .    | 9         |
| 3.4.2 Custom Classification Head . . . . .    | 9         |
| 3.5 Training Configuration . . . . .          | 10        |
| 3.6 Prediction Pipeline . . . . .             | 10        |
| 3.6.1 Image Prediction . . . . .              | 10        |
| 3.6.2 Video Prediction . . . . .              | 10        |
| <b>4 System Architecture</b>                  | <b>11</b> |

|          |                                         |           |
|----------|-----------------------------------------|-----------|
| 4.1      | Overall System Design . . . . .         | 11        |
| 4.2      | Technology Stack . . . . .              | 11        |
| 4.3      | Project Structure . . . . .             | 11        |
| 4.4      | System Workflow . . . . .               | 12        |
| 4.5      | User Interface Design . . . . .         | 12        |
| <b>5</b> | <b>Results and Testing</b>              | <b>14</b> |
| 5.1      | Model Performance . . . . .             | 14        |
| 5.2      | Threshold Analysis . . . . .            | 14        |
| 5.3      | Web Application Testing . . . . .       | 14        |
| 5.3.1    | Image Detection Testing . . . . .       | 14        |
| 5.3.2    | Video Detection Testing . . . . .       | 15        |
| 5.4      | System Limitations Identified . . . . . | 15        |
| <b>6</b> | <b>Conclusion and Future Work</b>       | <b>16</b> |
| 6.1      | Conclusion . . . . .                    | 16        |
| 6.2      | Future Work . . . . .                   | 16        |
|          | <b>References</b>                       | <b>18</b> |

# Abstract

The rapid proliferation of deepfake technology has posed significant threats to digital media authenticity, personal privacy, and public trust. Deepfakes — AI-generated manipulated media — are becoming increasingly indistinguishable from genuine content, making automated detection systems a critical necessity. This project presents **Deep-Scan**, a deepfake detection system that utilizes a frame-based image classification approach to identify manipulated images and videos.

The system employs a fine-tuned **EfficientNetB0** Convolutional Neural Network (CNN) as the primary detection model, trained on a curated dataset of 20,000 real and fake facial images derived from the FaceForensics++ dataset. Face detection is performed using the **Multi-task Cascaded Convolutional Networks (MTCNN)** algorithm, which extracts facial regions from each frame prior to classification. For video inputs, a majority voting mechanism aggregates per-frame predictions to produce a final verdict.

The system achieves an overall accuracy of **80–90%** on the test set and is deployed as a full-stack web application built with the **Django** framework, providing an intuitive user interface for uploading and analyzing both images and videos in real-time. The results demonstrate the feasibility of frame-based deepfake detection for practical deployment, while also highlighting limitations with respect to unseen deepfake generation techniques.

# Introduction

## 1.1 Background

In recent years, the advancement of deep learning techniques — particularly Generative Adversarial Networks (GANs) and autoencoder-based architectures — has enabled the creation of highly realistic synthetic media, commonly referred to as *deepfakes*. These AI-generated videos and images superimpose one person's face onto another's body or synthesize entirely new faces with remarkable realism.

While deepfake technology has legitimate applications in the entertainment industry, education, and accessibility, its malicious use has raised serious concerns including:

- Spread of political misinformation and propaganda
- Non-consensual intimate imagery
- Identity fraud and financial scams
- Erosion of trust in digital media

## 1.2 Problem Statement

The manual identification of deepfake content is practically infeasible given the enormous volume of digital media generated daily. Existing detection methods often require significant computational resources or fail to generalize across different deepfake generation techniques. There is a pressing need for accessible, accurate, and efficient automated deepfake detection tools.

## 1.3 Project Objectives

The primary objectives of this project are:

1. To design and implement a deepfake detection system capable of processing both image and video inputs.
2. To train a deep learning model on a balanced dataset of real and fake facial images.
3. To integrate MTCNN-based face detection with EfficientNetB0 classification.
4. To deploy the system as a user-friendly web application using the Django framework.

5. To evaluate the system's performance using standard classification metrics.

## 1.4 Scope and Limitations

### 1.4.1 What the System Does

- Accepts JPEG, PNG image files and MP4, AVI, MOV video files as input.
- Extracts faces using MTCNN and classifies them as REAL or FAKE.
- Applies majority voting across video frames to produce a final prediction.
- Displays confidence scores and detailed analysis statistics.

### 1.4.2 What the System Does Not Do

- Does not perform live or real-time streaming detection.
- Does not detect audio-based deepfakes.
- Does not reconstruct or reverse-engineer original faces.
- Does not detect GAN-synthesized faces outside the training distribution.

# Literature Review

## 2.1 Deepfake Generation Techniques

Deepfakes are primarily generated using two categories of deep learning approaches:

**Autoencoder-based methods** use encoder-decoder architectures trained to reconstruct faces. By sharing a common encoder and swapping decoders trained on different subjects, these models can convincingly transfer one person’s facial expressions onto another. FaceSwap and DeepFaceLab are prominent examples of this approach.

**GAN-based methods** such as StyleGAN and ProGAN generate entirely synthetic faces that do not correspond to any real person. These methods produce high-resolution, photorealistic faces and are widely used in datasets like FFHQ.

## 2.2 Deepfake Detection Approaches

The detection of deepfake content has been an active research area. Several key approaches have been proposed in literature:

### 2.2.1 CNN-Based Detection

Rossler et al. (2019) introduced the FaceForensics++ benchmark and demonstrated that CNN-based classifiers could effectively detect compression artifacts and manipulation traces left by face-swap algorithms. Their work established the foundation for frame-level deepfake detection.

### 2.2.2 Transfer Learning Approaches

Pre-trained models such as VGG, ResNet, and EfficientNet have been widely adopted for deepfake detection through transfer learning. Tan and Le (2019) demonstrated that EfficientNet achieves state-of-the-art performance on image classification tasks with significantly fewer parameters than comparable architectures.

### 2.2.3 Temporal Analysis

Approaches leveraging temporal inconsistencies in videos — such as unnatural blinking patterns or head movements — have also been explored. However, these methods require

longer video sequences and are computationally intensive.

### 2.2.4 Face-Region Focused Detection

Several studies have shown that focusing detection on the facial region, rather than the entire frame, significantly improves accuracy. This observation motivated our use of MTCNN for face extraction prior to classification.

## 2.3 Selected Approach Justification

Based on the literature, our system adopts a **frame-based, face-region focused** detection approach using EfficientNetB0, which offers the best balance between accuracy and computational efficiency for a semester-level implementation.

# Methodology

## 3.1 Dataset

The model was trained on a curated dataset of **20,000 images** (10,000 real and 10,000 fake), derived from the FaceForensics++ dataset. The dataset was split as follows:

Table 3.1: Dataset Split

| Split        | Real Images   | Fake Images   | Total         |
|--------------|---------------|---------------|---------------|
| Training     | 5,000         | 5,000         | 10,000        |
| Validation   | 2,500         | 2,500         | 5,000         |
| Testing      | 2,500         | 2,500         | 5,000         |
| <b>Total</b> | <b>10,000</b> | <b>10,000</b> | <b>20,000</b> |

The dataset was balanced to prevent class bias, with equal representation of real and fake samples in each split. Random sampling was applied to avoid selection bias.

## 3.2 Data Preprocessing

The following preprocessing steps were applied to all images before training:

1. **Face Detection:** MTCNN was used to detect and crop the facial region from each image, with a 20-pixel margin around the detected bounding box.
2. **Resizing:** All face crops were resized to  $224 \times 224$  pixels to match EfficientNetB0’s expected input dimensions.
3. **Preprocessing:** EfficientNet’s built-in preprocessing function (`preprocess_input`) was applied, which performs channel-wise normalization.

## 3.3 Data Augmentation

To improve generalization and reduce overfitting, the following augmentation techniques were applied to the training set:

Table 3.2: Data Augmentation Techniques

| Technique             | Parameters         |
|-----------------------|--------------------|
| Rotation              | $\pm 20^\circ$     |
| Width Shift           | $\pm 15\%$         |
| Height Shift          | $\pm 15\%$         |
| Horizontal Flip       | Enabled            |
| Zoom                  | $\pm 15\%$         |
| Brightness Adjustment | Range $[0.8, 1.2]$ |

## 3.4 Model Architecture

### 3.4.1 Base Model: EfficientNetB0

EfficientNetB0 was selected as the backbone architecture due to its:

- Superior accuracy-to-parameter ratio compared to VGG and ResNet
- Compound scaling methodology that uniformly scales depth, width, and resolution
- Strong performance on transfer learning tasks

### 3.4.2 Custom Classification Head

The pre-trained EfficientNetB0 was extended with the following classification layers:

Table 3.3: Model Architecture

| Layer                  | Output Shape | Parameters       |
|------------------------|--------------|------------------|
| EfficientNetB0 (base)  | (7, 7, 1280) | 4,049,571        |
| Global Average Pooling | (1280)       | 0                |
| Dense (ReLU)           | (256)        | 327,936          |
| Dropout (0.5)          | (256)        | 0                |
| Dense (ReLU)           | (128)        | 32,896           |
| Dropout (0.3)          | (128)        | 0                |
| Dense (Sigmoid)        | (1)          | 129              |
| <b>Total</b>           |              | <b>4,410,532</b> |

## 3.5 Training Configuration

Table 3.4: Training Hyperparameters

| Parameter      | Value                              |
|----------------|------------------------------------|
| Optimizer      | Adam                               |
| Learning Rate  | $1.25 \times 10^{-5}$              |
| Loss Function  | Binary Cross-Entropy               |
| Batch Size     | 16                                 |
| Metrics        | Accuracy, Precision, Recall        |
| Early Stopping | Monitored on <code>val_loss</code> |
| LR Reduction   | ReduceLROnPlateau                  |

## 3.6 Prediction Pipeline

### 3.6.1 Image Prediction

1. Load and decode the uploaded image.
2. Apply MTCNN face detection with 20px margin.
3. If no face detected, use the full image as input.
4. Apply EfficientNet preprocessing and resize to  $224 \times 224$ .
5. Feed through the model to obtain probability score  $p \in [0, 1]$ .
6. Apply threshold:  $p > 0.45 \Rightarrow \mathbf{REAL}$ ,  $p \leq 0.45 \Rightarrow \mathbf{FAKE}$ .
7. Return label and confidence score.

### 3.6.2 Video Prediction

1. Extract frames from the video at every 5th frame.
2. For each frame, apply MTCNN face detection.
3. Classify each face crop using the model.
4. Aggregate predictions: compute mean probability across all frames.
5. Apply video threshold:  $\text{mean} > 0.30 \Rightarrow \mathbf{REAL}$ , otherwise  $\mathbf{FAKE}$ .
6. Return verdict, confidence, and frame-level statistics.

# System Architecture

## 4.1 Overall System Design

The DeepScan system follows a three-tier web application architecture consisting of:

- **Presentation Layer:** Django HTML templates with CSS/JavaScript frontend
- **Application Layer:** Django views handling request routing and business logic
- **Model Layer:** TensorFlow/Keras EfficientNetB0 model with MTCNN face detector

## 4.2 Technology Stack

Table 4.1: Technology Stack

| Component         | Technology              |
|-------------------|-------------------------|
| Backend Framework | Django 4.2 (Python)     |
| Deep Learning     | TensorFlow / Keras 3.13 |
| Face Detection    | MTCNN                   |
| Image Processing  | OpenCV                  |
| Frontend          | HTML5, CSS3, JavaScript |
| Database          | SQLite                  |
| Model Format      | Keras Native (.keras)   |

## 4.3 Project Structure

The project is organized as follows:

Listing 4.1: Project Directory Structure

```
1 deepfake_detector/  
2 |-- manage.py  
3 |-- requirements.txt  
4 |-- deepfake_detector/  
5 |   |-- settings.py  
6 |   |-- urls.py  
7 |   '-- wsgi.py  
8 |-- detector/
```

```
9 |     |-- ml_model.py
10 |     |-- views.py
11 |     |-- urls.py
12 |     '-- model/
13 |         '-- efficientnet_best.keras
14 |-- templates/
15 |     |-- base.html
16 |     '-- detector/
17 |         |-- index.html
18 |         '-- result.html
19 '-- media/uploads/
```

## 4.4 System Workflow

The end-to-end workflow of the system is as follows:

1. User accesses the web interface at `http://127.0.0.1:8000`
2. User uploads an image or video file via the drag-and-drop interface
3. Django view receives the file via HTTP POST request
4. File is saved to `media/uploads/` with a unique UUID filename
5. File type is determined by extension
6. `ml_model.py` is invoked for prediction:
  - MTCNN detects and crops the facial region
  - EfficientNet preprocesses and classifies the face
  - For videos: majority voting aggregates frame-level predictions
7. Results are passed to the result template
8. User sees: verdict (REAL/FAKE), confidence score, and analysis statistics

## 4.5 User Interface Design

The frontend was designed with a dark, modern aesthetic featuring:

- **Upload Page:** Drag-and-drop file upload zone with format validation, file preview, and animated loading overlay during analysis

- **Result Page:** Large verdict display with color coding (green for REAL, red for FAKE), animated confidence progress bar, media preview, and detailed statistics panel
- **Responsive Design:** Grid-based layout adapting to different screen sizes
- **Typography:** Orbitron (display) and Fira Code (body) font families

# Results and Testing

## 5.1 Model Performance

The EfficientNetB0 model was evaluated on the held-out test set of 5,000 images. The following performance metrics were recorded:

Table 5.1: Model Evaluation Metrics

| Metric           | Value                |
|------------------|----------------------|
| Overall Accuracy | 80–90%               |
| Real Recall      | $\geq 85\%$          |
| Fake Recall      | $\geq 80\%$          |
| Precision        | $\geq 82\%$          |
| Loss Function    | Binary Cross-Entropy |

## 5.2 Threshold Analysis

The decision threshold was tuned to maximize balanced accuracy between REAL and FAKE recall. The final thresholds selected were:

- **Image threshold:** 0.45 — maximizes balanced accuracy for still images
- **Video threshold:** 0.30 — accounts for frame-level variance in videos

## 5.3 Web Application Testing

### 5.3.1 Image Detection Testing

The system was tested with the following categories of images:

Table 5.2: Image Testing Results

| Image Type            | Source                 | Result                               |
|-----------------------|------------------------|--------------------------------------|
| Real portraits        | Personal photos        | Correctly classified as REAL         |
| FaceForensics++ fakes | Training dataset       | Correctly classified as FAKE         |
| GAN-generated faces   | thispersondoesnotexist | Misclassified (outside distribution) |

### 5.3.2 Video Detection Testing

Video files were tested using the frame-sampling pipeline:

Table 5.3: Video Testing Results

| Video Type          | Verdict | Avg. Confidence |
|---------------------|---------|-----------------|
| Real person videos  | REAL    | $\geq 80\%$     |
| Face-swap deepfakes | FAKE    | $\geq 75\%$     |

## 5.4 System Limitations Identified

Through testing, the following limitations were identified:

1. **GAN-generated faces:** The model fails to detect fully synthetic faces (e.g., from StyleGAN) as it was trained only on face-swap deepfakes. This is a known generalization challenge in deepfake detection literature.
2. **No face detected:** When MTCNN fails to detect a face (e.g., blurry or low-resolution images), the full image is used as fallback, reducing accuracy.
3. **Compression artifacts:** Heavily compressed images or videos may reduce prediction confidence.
4. **Processing time:** Video analysis takes longer due to per-frame inference, particularly for videos with many faces.

# Conclusion and Future Work

## 6.1 Conclusion

This project successfully developed and deployed **DeepScan**, a deepfake detection system using frame-based image classification. The key achievements of this project include:

- Successfully trained an EfficientNetB0 model achieving **80–90% accuracy** on the FaceForensics++ test set.
- Implemented a robust prediction pipeline incorporating MTCNN face detection, EfficientNet classification, and majority voting for videos.
- Deployed a fully functional Django web application with an intuitive, dark-themed user interface supporting both image and video uploads.
- Demonstrated the viability of frame-based deepfake detection as a practical, computationally accessible approach.

The system effectively detects face-swap deepfakes — the type of manipulation it was trained on — while highlighting an important finding: models trained on specific deepfake techniques do not automatically generalize to other generation methods. This observation is consistent with findings in deepfake detection literature.

## 6.2 Future Work

Several directions can enhance this system in future iterations:

1. **Multi-technique Training:** Extend the training dataset to include GAN-generated faces (StyleGAN, ProGAN) to improve generalization across deepfake types.
2. **Temporal Analysis:** Incorporate LSTM or 3D CNN layers to exploit temporal inconsistencies across video frames for improved video detection.
3. **Real-time Detection:** Implement WebSocket-based streaming detection for live video analysis.
4. **Explainability:** Integrate Grad-CAM visualization to highlight the facial regions that influenced the model’s decision.
5. **Audio Deepfake Detection:** Extend the system to detect voice cloning and audio manipulation alongside visual deepfakes.

6. **Model Ensemble:** Combine predictions from multiple models (EfficientNet + CNN) to improve robustness.
7. **Mobile Deployment:** Convert the model to TensorFlow Lite for deployment on mobile devices.

# References

1. Rossler, A., Cozzolino, D., Verdoliva, L., Riess, C., Thies, J., & Nießner, M. (2019). *FaceForensics++: Learning to Detect Manipulated Facial Images*. Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV).
2. Tan, M., & Le, Q. V. (2019). *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks*. Proceedings of the 36th International Conference on Machine Learning (ICML).
3. Zhang, K., Zhang, Z., Li, Z., & Qiao, Y. (2016). *Joint Face Detection and Alignment Using Multi-task Cascaded Convolutional Networks*. IEEE Signal Processing Letters, 23(10), 1499–1503.
4. Li, Y., Chang, M. C., & Lyu, S. (2018). *In Ictu Oculi: Exposing AI Created Fake Videos by Detecting Eye Blinking*. IEEE International Workshop on Information Forensics and Security (WIFS).
5. Chollet, F. (2017). *Keras: Deep Learning for Humans*. GitHub. <https://github.com/keras-team/keras>
6. Django Software Foundation. (2023). *Django: The Web Framework for Perfectionists with Deadlines*. <https://www.djangoproject.com>
7. Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., & Bengio, Y. (2014). *Generative Adversarial Nets*. Advances in Neural Information Processing Systems (NeurIPS).
8. Bradski, G. (2000). *The OpenCV Library*. Dr. Dobb’s Journal of Software Tools.